

A Tutorial on the Mimetic Operators Library Enhanced (MOLE) with a brief introduction to Mimetic Differences

M. A. Dumett¹

¹Computational Science Research Center
San Diego State University

MOLE Tutorial, May 12 2026

MOLE

Tutorial Overview

- Mimetic differences
- MOLE
- Examples

MOLE

Mimetic Differences

- Mimetic differences aims to construct a discrete vector calculus
- Mimetic differences constructs discrete analogs of $\nabla, \nabla \cdot, \nabla \times, \nabla^2$
- Mimetic differences can build mimetic numerical schemes with them
- Aim not only for accuracy but to replicate symmetries, conserved quantities, fundamental identities, etc.
- Can achieve uniform accuracy over the whole domain

Mimetic Differences

- 1 Use a staggered grid
- 2 2D & 3D operators constructed by Kronecker products
- 3 Operators accurately satisfy the extended Gauss' divergence theorem
- 4 They reproduce vector calculus identities (also the product rule)
- 5 It has been demonstrated that they preserve mass, energy and momenta for systems of conservation laws with smooth data
- 6 They have been successfully used in applications with curvilinear, non-uniform and unstructured grids

MOLE

Review of Finite Differences

1D Poisson's Equation with Dirichlet BC

$$\begin{aligned}-u''(x) &= f(x), \quad x \in (0,1) \\ u(0) &= 0, \\ u(1) &= 0\end{aligned}$$

FD discretization steps:

- Let $u(x)$ be the true solution
- Consider n cells or subintervals of size $h = \frac{1}{n}$ in $[0, 1]$
- There are n interior nodes $x_j = jh$, $j = 1, \dots, n-1$
- Define two ghost points $x_0 = 0$ and $x_n = 1$ for the BC
- Let $u_j = u(x_j)$ and $f_j = f(x_j)$
- u_j , $j = 1, \dots, n-1$ are the unknowns

FD Review: 1D Poisson's Equation with Dirichlet BC

- After approximating $\frac{d^2}{dx^2}$ by D^2 one gets the FD scheme

$$-D^2 u_j = \frac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} = f_j, \quad j = 1, \dots, n-1 \quad (1)$$

Define

$$U_h = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-1} \end{bmatrix}, \quad F = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_{n-1} \end{bmatrix}, \quad K = \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & -1 & \\ & & \ddots & \ddots & \ddots \\ & & & -1 & 2 & -1 \\ & & & & -1 & 2 \end{bmatrix}$$

The FD scheme in matrix vector form

$$\frac{1}{h^2} K U_h = F$$



Mimetic Differences

1D Poisson's Equation with Dirichlet BC

$$\begin{aligned}-u''(x) &= f(x), & x \in (0,1) \\ u(0) &= 0, \\ u(1) &= 0\end{aligned}$$

MD discretization steps:

- Let $u(x)$ be the true solution
- Consider n cells or subintervals of size $h = \frac{1}{n}$ in $[0, 1]$
- Cell faces $x_j = jh$, $j = 0, 1, \dots, n$
- Cell centers $x_j = jh$, $j = 0, 1/2, \dots, n - 1/2, n$.
- Let $u_j = u(x_j)$ and $f_j = f(x_j)$, $j = 0, 1/2, \dots, n - 1/2, n$
- u_j , $j = 0, 1/2, \dots, n - 1/2, n$ are the unknowns

Mimetic: 1D Poisson's Equation with Dirichlet BC

- After approximating $\frac{d^2}{dx^2}$ by $L^{(2)}$ one gets the mimetic scheme, where

$$L = L^{(2)} = D^{(2)} G^{(2)} \quad (2)$$

Define

$$U_h = \begin{bmatrix} u_0 \\ u_{1/2} \\ \vdots \\ u_{n-1/2} \\ u_n \end{bmatrix}, F = \begin{bmatrix} f_0 \\ f_{1/2} \\ \vdots \\ f_{n-1/2} \\ f_n \end{bmatrix}, L^{(2)} = \begin{bmatrix} 1 & & & & & \\ 8/3 & -4 & 1/3 & & & \\ & -1 & 2 & -1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & -1 & 2 & -1 \\ & & & & -1/3 & 4 & -8/3 \\ & & & & & & 1 \end{bmatrix}$$

The mimetic difference scheme in matrix vector form (after adding boundary conditions) is

$$\frac{1}{h^2} L^{(2)} U_h = F$$



MOLE Operators

MOLE is an implementation of mimetic differences operators

$$D = \nabla \cdot + \mathcal{O}(h^k) \quad \text{first and last rows are zero}$$

$$G = \nabla + \mathcal{O}(h^k)$$

$$Q = \mathcal{O}(h^{k+1}) \quad \text{quadrature for scalar fields}$$

$$P = \mathcal{O}(h^{k+1}) \quad \text{quadrature for vector fields}$$

$$I^{c \rightarrow f} = \mathcal{O}(h^k) \quad \text{interpolation from cell centers to faces}$$

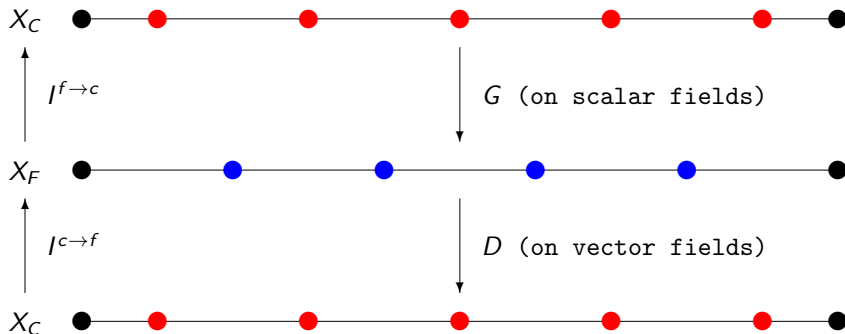
$$I^{f \rightarrow c} = \mathcal{O}(h^k) \quad \text{interpolation from faces to cell centers}$$

$$C = \nabla \times + \mathcal{O}(h^k)$$

$$L = \nabla^2 + \mathcal{O}(h^k) \quad \text{first and last rows are zero}$$



1D Staggered-Grid

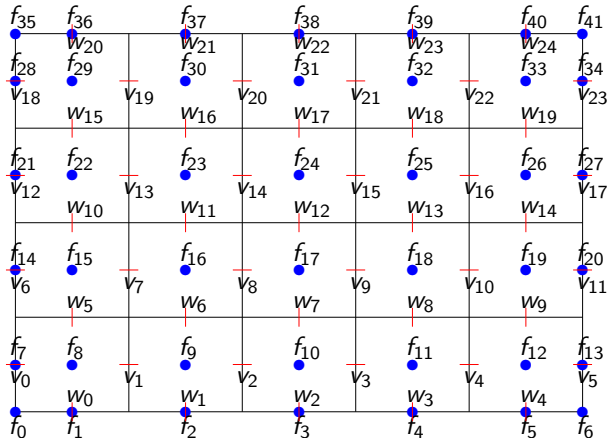


X_C centers, X_F faces

Notice that the cell centers grid is non-uniform



2D Staggered-Grid



A 5×4 staggered grid for scalar f and vector (v, w) fields

Observe that nodes and faces are different

$$D : V \cup W \rightarrow F \text{ and } G : F \rightarrow V \cup W$$



Preservation and scheme example: 2D Euler equations

$$\frac{\partial q}{\partial t} + \nabla \cdot f = 0,$$

with

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{pmatrix} = \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ E \end{pmatrix}, \quad f_1 = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho u v \\ (E + p)u \end{pmatrix}, \quad f_2 = \begin{pmatrix} \rho v \\ \rho u v \\ \rho v^2 + p \\ (E + p)v \end{pmatrix},$$

- ρ is the density, u the horizontal velocity, v the vertical velocity, E the total energy. γ fluid-dependent constant (atmospheric gases $\gamma = \frac{7}{5}$).

The equations are closed by the ideal gas law

$$p = (\gamma - 1) \left(E - \frac{1}{2} \rho(u^2 + v^2) \right).$$

MOLE

Discrete Analog

- $Q = [Q_1; Q_2; Q_3; Q_4]$
- Semi-discrete mimetic scheme:

$$Q_t = -D I^{c \rightarrow f} F(Q)$$

$$F = \begin{pmatrix} \text{diag}(Q_2) & \text{diag}(Q_3) \\ \text{diag}(Q_2^2./Q_1 + P) & \text{diag}(Q_2.*Q_3)./Q_1 \\ \text{diag}(Q_2.*Q_3)./Q_1 & \text{diag}(Q_3^2./Q_1 + P) \\ \text{diag}((Q_4 + P).*Q_2./Q_1) & \text{diag}((Q_4 + P).*Q_3./Q_1) \end{pmatrix}$$

$$P = (\gamma - 1) \text{diag} \left(Q_4 - \frac{1}{2} (Q_2^2 + Q_3^2) ./ Q_1 \right)$$

- Use a time discretization scheme
- Add boundary conditions to the resulting operator

- MOLE is a sparse implementation of these discrete analogs
- MOLE is a C++ & MATLAB/Octave modular library
- Focus on MATLAB/Octave
- MOLE examples



MOLE download/clone instructions

- ➊ Go to **MOLE**
- ➋ Fork the repository
- ➌ Download or Clone the repository to your local computer
 - For downloading
 - ➊ Press the **Code** button
 - ➋ Choose option Download ZIP
 - ➌ Extract the ZIP file into the directory of your choice
 - For cloning
 - ➊ Open terminal
 - ➋ Clone the repository to your local computer (follow git instructions)
 - ➌ Enter mole folder
- ➍ Open up one of the examples located in examples/matlab_octave or proceed to install the C++ version of the library following the instructions provided in **README**

The MOLE logo is displayed in a blue-bordered box. It consists of the word "MOLE" in a bold, blue, sans-serif font.

PDE Boundary Conditions

$$u(x) = g(x), \quad x \in \partial U \quad \text{Dirichlet}$$

$$\frac{\partial u}{\partial \vec{n}} = h(x), \quad x \in \partial U \quad \text{Neumann}$$

$$\alpha u + \beta \frac{\partial u}{\partial \vec{n}} = k(x), \quad x \in \partial U \quad \text{Robin}$$

MOLE code steps

- 1 Staggered grid
- 2 Using discrete analogs for $\nabla, \nabla \cdot, \nabla \times, \nabla^2$
- 3 Need to convert arrays of unknowns into vectors (ordering)
- 4 Elliptic examples: Adding boundary conditions
- 5 Time discretizations
- 6 Time loops: Adding boundary conditions

MOLE Standard Notation

For some parameters

- k : the degree of accuracy ($k = 2, 4, 6, 8$)
- $m, n, o \geq 2k + 1$: the number of cells along axis x, y, z , respectively

Elliptic Examples

- ① elliptic1DHomogeneousDirichlet
- ② elliptic1DLeftNeumannRightRobin
- ③ sturmLiouvilleHermite
- ④ elliptic2DXPerYDirichlet
- ⑤ elliptic3DXDirichletYPeriodicZPeriodic

Parabolic PDE with Neumann BC

In the domain $D = \Omega \times (0, T)$, where Ω is the rectangle $(0, l) \times (0, h)$, consider the time-dependent conductivity the 2D heat equation

$$u_t = (1/50)(1+t) \nabla^2 u, \quad (x, y, t) \in D,$$

subject to the initial condition

$$u(x, y, 0) = \exp(x - y + 1/50), \quad x \in [0, l], y \in [0, h],$$

and the Neumann boundary conditions

$$\partial u / \partial n(0, y, t) = \exp((1/50)(1+t)^2 - y)$$

$$\partial u / \partial n(l, y, t) = \exp(1 + (1/50)(1+t)^2 - y)$$

$$\partial u / \partial n(x, 0, t) = \exp((1/50)(1+t)^2 + x)$$

$$\partial u / \partial n(x, h, t) = \exp(-1 + (1/50)(1+t)^2 + x)$$

with analytical solution given by

$$u(x, y, t) = \exp((1/50)(1+t)^2 + x - y)$$

Consider $l = h = T = 1$



Explicit Matlab snippet (Thanks to Ben Wagner)

```
i = 0;
```

```
while dt*i < 1
```

```
    left = l(Y(1,:), dt*i)';
```

```
    right = r(Y(end,:), dt*i)';
```

```
    bottom = b(X(:,1), dt*i);
```

```
    top = t(X(:,end), dt*i);
```

```
    [Lt, uab] = addScalarBC2D(L, ua, k, m, dx, n, dy, ...
```

```
        dc, nc, { left(2:end-1); right(2:end-1); bottom; top } );
```

```
    ua = ua + dt*a(dt*i)*Lt*uab;
```

```
    i = i + 1;
```

```
end
```

Implicit Matlab snippet (Thanks to Xuerui Shi)

```
for step = 1 : Nsteps

    %update interior DOFs
    U_new = U;
    U_new(idx_int) = U(idx_int) + dt * ((1+t_now)/50) * (L_int * U);

    %update boundary DOFs
    t_next = t_now + dt;

    v = {arrayfun(@(y) -exp(c(t_next) - y),      y_int_pts); ...
          arrayfun(@(y)  exp(1+c(t_next) - y),      y_int_pts); ...
          arrayfun(@(x)  exp(c(t_next) + x),        x_all_pts); ...
          arrayfun(@(x) -exp(-1+c(t_next) + x),      x_all_pts)};

    [~, b_next] = addScalarBC2D(L, zeros(N,1), k, m, dx, n, dy, dc_bc, nc_bc, v);
    A_bb = A_bc_rows(:, idx_bc);
    A_bi = A_bc_rows(:, idx_int);
    rhs_bc = b_next(idx_bc) - A_bi * U_new(idx_int);
    U_new(idx_bc) = A_bb \ rhs_bc;

    U = U_new;
    t_now = t_next;
end
```

Implicit/Explicit Matlab snippet (Thanks to Dalston Karto)

```
for n = 0:nt
    tnow = n*dt;
    bcl = zeros(ny+2,1); bcr = zeros(ny+2,1); bcb = zeros(nx+2,1); bct = zeros(nx+2,1);

    for j = 1:ny+2
        bcl(j) = mu1(ygrid(j), tnow); % left: x = 0
        bcr(j) = mu2(ygrid(j), tnow); % right: x = 1
    end

    for i = 1:nx+2
        bcb(i) = -nu1(xgrid(i), tnow); % bottom: y = 0
        bct(i) = -nu2(xgrid(i), tnow); % top: y = 1
    end

    bcl = bcl(2:end-1,1); bcr = bcr(2:end-1,1); v = {bcl; bcr; bcb; bct};

    Uvec = reshape(U, [], 1);

    [L0, Uvec] = addScalarBC2D(L, Uvec, k, nx, dx, ny, dy, dc, nc, v);

    alpha_n = a(tnow);

    switch method
        case "explicit"
            A = I + dt*alpha_n*L0;
            Uvec = A * Uvec;

        case "implicit"
            A = I - dt*alpha_n*L0;
            Uvec = A \ Uvec;
    end

    U = reshape(Uvec, nx+2, ny+2);
end
```

Wave Examples

1 wave1DTimeVaryingBC

$$\begin{aligned}u_{tt} &= c^2 u_{xx}, & 0 < x < M, & & 0 < t < T \\u(0, t) &= \sin(\pi x), & u_t(0, t) &= 0 & \text{Initial conditions} \\u(0, t) &= 0, & u(M, t) &= 0 & \text{Boundary conditions}\end{aligned}$$

2 1D Serre-Green-Naghdi model with constant bathymetry

$$\begin{aligned}h_t + (hu)_x &= 0, \\(hu)_t + \left(hu^2 + \frac{1}{2}gh^2\right)_x &= 0 \\-\frac{1}{3}h^2u_{xx} - hh_xu_x + u &= q\end{aligned}$$

Thanks to Christos Papoutsellis

MOLE

Handling boundaries without the addScalarBC feature

- Wound Healing (see video in MOLE page)
- 1D Maxwell equations